Python Exercises

The following exercises are to be prepared by the fifth session of the course (16th of November). Until this session, mark all exercises which you are able to present as such on our e-learning platform. Also make sure that you have submitted your work packaged in a `zip` file, as described in Appendix A.
A student will be picked at random to be interviewed about their solution. If there are clear doubts that the student can explain the presented work, no exercise of this sheet will be counted towards the final grade. The solution will be further discussed with the plenum, for a common learning experience.

For exercises 1 and 2, the modules of the Python standard library are all you need. From exercise 3 onward, use `numpy`; use other third-party libraries such as `pandas` or `sklearn` only where explicitly mentioned.

As for the first sheet, put all of your code into one package, this time called `aaip2`. In this package, each of the following exercises gets a module called `ex1`, `ex2`, `ex3` and so on. For every exercise, add a script `scripts/run_ex<i>.py` outside the package that imports your module and demonstrates it. Appendix A summarizes the project structure.

1 Command-Line Tool Using `sys.argv`

Write a command-line tool for simple operations on text files. Implement the operations as functions in `aaip2/ex1.py`, and read the command-line arguments with `sys.argv` in the script `scripts/run_ex1.py`. The script takes the following arguments:

- the name of a text file,
- the operation to perform: `count_words`, `count_lines` or `search_word`, and
- for `search_word` only, the word to search for.

Depending on the operation, the script prints the number of words in the file, the number of lines in the file, or all lines that contain the given word. If the file does not exist, or if arguments are missing or invalid, the script reports this with a clear error message.

Add a small text file `example.txt` of your choice to your project, so that the following calls work:

```
python scripts/run_ex1.py example.txt count_words
python scripts/run_ex1.py example.txt count_lines
python scripts/run_ex1.py example.txt search_word Python
```

Here and below, `python` stands for the Python of your project's environment (see Appendix A).

2 Command-Line Tool Using `argparse`

Next, write a more advanced command-line tool for file management, this time using the `argparse` module. Implement the file operations in `aaip2/ex2.py` and the command-line interface in `scripts/run_ex2.py`. The tool supports three subcommands:

- `copy` copies a file from a source to one or more destinations,
- `move` moves a file from a source to a destination, and
- `rename` renames a file.

The source and the destination are passed with the options `-src` and `-dst`, in any order. Use `argparse` such that missing arguments lead to clear error messages, and that `-h` or `-help` prints a detailed help text. After each operation, the tool confirms it with a short message, e.g. `File copied successfully`. For the file operations themselves, you may use the `shutil` module. For example:

```
python scripts/run_ex2.py copy --src source.txt --dst copy_1.txt copy_2.txt
python scripts/run_ex2.py move --dst destination.txt --src source.txt
python scripts/run_ex2.py rename --src old_name.txt --dst new_name.txt
```

3 Gentle Introduction to `NumPy`

This exercise makes you comfortable with `numpy` arrays: how to create them, their shapes and data types, slicing, and broadcasting. Try to solve the tasks on your own first. If you get stuck, have a look at the numpy introduction for absolute beginners. Write small functions for the tasks in `aaip2/ex3.py`, and a script `scripts/run_ex3.py` that calls them and prints the results. The script prints the shapes and data types from 3.1, the selections and counts from 3.2, and the column means before and after centering from 3.3. It runs without any additional files:

```
python scripts/run_ex3.py
```

3.1 Arrays, shapes and data types

Create

- the 1D array $[0, 1, 2, 3, 4]$,
- the 2D array $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$,
- the 3×3 identity matrix, and
- a 4×3 array from the Python list `[[0,1,2], [3,4,5], [6,7,8], [9,10,11]]`. Reshape it to $(3, 4)$ and flatten it back to 1D.

For each of these arrays, print its `shape` and `dtype`.

3.2 Slicing and masking

Let M be the array $[0, 1, \ldots, 11]$, reshaped to $(3, 4)$.

- Select the second column as a 1D array.
- Select the last row in reverse order.
- Select the top-left 2×2 block.
- Create a boolean mask for the entries that are larger than the mean of M. Count these entries, and compute their mean.

3.3 Broadcasting without loops

Using the same M,

- add a 1D array v of length 4 to each row of M, and
- subtract the *column means* of M from M, so that every column of the result has (approximately) mean zero. Verify this by printing the column means.

4 Ridge Regression on the Prostate Data

We return to the prostate cancer data from the lecture, taken from the book by Hastie, Tibshirani and Friedman. The goal is to predict the logarithm of the prostate-specific antigen (`lpsa`) from the eight clinical predictors `lcavol`, `lweight`, `age`, `lbph`, `svi`, `lcp`, `gleason` and `pgg45`.
Ordinary least squares (OLS) chooses the weights w that minimize the sum of squared residuals. *Ridge* regression adds an L_2 penalty on the weights, which reduces the variance of the estimate and helps with strongly correlated features (multicollinearity):

$$\min_{w,b} \|y - (b\mathbf{1} + Xw)\|_2^2 \; + \; \lambda \|w\|_2^2, \qquad \lambda \geq 0. \tag{1}$$

The intercept b is **not** penalized. For a fixed λ, the normal equations become

$$(X^\top X + \lambda I)\, w \;=\; X^\top (y - b\mathbf{1}). \tag{2}$$

As λ increases, the weights shrink towards zero: the estimate becomes biased, but its variance typically decreases. Since the penalty depends on the scale of the features, we standardize the features before fitting, using statistics of the training data only.

In practice, compute the mean μ and the standard deviation σ of the training features (the sample standard deviation with `ddof=1`, as in the lecture; replace zeros in σ by 1). Standardize the features, $X_s = (X - \mu)/\sigma$, and center the targets, $y_c = y - \bar{y}$, where $\bar{y}$ is the mean of the training targets. Then

$$w_s = (X_s^\top X_s + \lambda I)^{-1} X_s^\top y_c, \quad w = w_s/\sigma, \quad b = \bar{y} - \mu^\top w, \tag{3}$$

and the predictions on the raw features are $\hat{y} = b + Xw$.

Copy the file `prostate.data` from the lecture (`lectures/lecture04/notebooks/` in the course repository) into the folder `data/` of your project. Its column `train` (`T`/`F`) contains a predefined split. As in the lecture, treat the rows with `T` as the training set, and split the rows with `F` randomly into a validation and a test set of equal size. Besides `numpy` and the standard library, you may use `pandas` for this exercise. Your script `scripts/run_ex4.py` runs the whole pipeline below and prints λ^*, a summary of the validation curve, the test MSEs of ridge regression and OLS, and a short comment on the results:

```
python scripts/run_ex4.py
```

4.1 Load and preprocess

Read `prostate.data`, preprocess it and split it as we did in the lecture, with `lpsa` as the target y and the eight predictors as X. Standardize the features with our routine from the lecture as well. You are free to reuse any functions from the lecture notebook.

4.2 Ridge weights

Implement equation (3) to obtain the ridge weights, and make predictions on the validation set.

4.3 Validation curve

Fit ridge regression on the training data for a logarithmic grid of values λ between 10^{-6} and 10^{2} (see, e.g., `np.logspace`), and compute the mean squared error (MSE) on the validation set for each λ. Determine the value λ^* with the smallest validation error.

4.4 Refit and test

Refit ridge regression with λ^* on the training set, and report its MSE on the test set. For comparison, also report the test MSE of OLS, with the same preprocessing but without a penalty. You can check your results against `sklearn`'s `Ridge`. Note that `Ridge` does not standardize the features itself: fit it on X_s, then its weights match w_s for `alpha` $= \lambda$.

4.5 Discussion

In 5 to 7 sentences, explain how standardization affects ridge regression, why the intercept is not penalized, and why the test MSE of ridge regression may be higher or lower than that of OLS.

4.6 Think

What happens when you replace the L_2 penalty on the weights by an L_1 penalty?

A Structuring and Handing in Your Package

Set up, install and hand in your project as described in Appendix A of the first sheet, with the following changes. Name the root folder `sheet_02_<id>` and the zip file `sheet_02_<id>.zip`, with `<id>` as for the first sheet (e.g. `sheet_02_mmueller`):

```
sheet_02_<id>/              <- the root folder you hand in
    pyproject.toml          <- describes the package (see below)
    example.txt             <- your text file for exercise 1
    data/
        prostate.data       <- copied from the lecture, for exercise 4
    aaip2/                  <- the package
        __init__.py         <- marks aaip2 as a package, may be empty
        ex1.py              <- one module per exercise
        ex2.py
        ex3.py
        ex4.py
    scripts/                <- scripts "outside the scope of the package"
        run_ex1.py
        run_ex2.py
        run_ex3.py
        run_ex4.py
```

The file `pyproject.toml` differs from the first sheet in two places. Since your code uses `numpy`, the list `dependencies` names it, so that the installation installs it as well. If you use `pandas` or `sklearn`, add `"pandas"` or `"scikit-learn"` to this list. Furthermore, the table `[tool.setuptools]` names the package explicitly. Otherwise, `setuptools` would also take the folder `data/` for a package and refuse to build.

```toml
[build-system]
requires = ["setuptools>=64"]
build-backend = "setuptools.build_meta"

[project]
name = "aaip2"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = ["numpy"]

[tool.setuptools]
packages = ["aaip2"]
```

In the commands on this sheet, `python` stands for the Python of your project's environment. With uv, prefix the commands by `uv run`, e.g.

```
uv run python scripts/run_ex1.py example.txt count_words
```

On the university lab PCs, install your package as for the first sheet, and call the Python of its virtual environment directly, e.g.

```
.venv\Scripts\python scripts\run_ex1.py example.txt count_words
```

To find `prostate.data` no matter where you run a script from, build its path relative to the script:

```python
from pathlib import Path

# scripts/run_ex4.py -> scripts/ -> root folder -> data/prostate.data
data_path = Path(__file__).parent.parent / "data" / "prostate.data"
```

Before zipping, delete the folders `.venv/`, `aaip2.egg-info/` and all `__pycache__/` folders, and check your zip file by unpacking it into an empty folder, installing it there and running all scripts.